DS701 Homework 5: Bias–Variance, Trees, and Ensembles

Released Wed Oct 7 · due Tue Oct 13, 11:59 pm on Gradescope

SOLUTIONS

Problem 1: Bias and variance, made concrete

Practices: turning "high bias / high variance" from words into numbers, and reading a learning curve.

We want to predict the value of some function at a single test input x_0, whose true value is $f(x_0) = 10$. Two models were each trained four times, on four independent training sets drawn from the same distribution, and asked to predict at x_0:

training set	1	2	3	4
Model A (very simple)	7	8	8	9
Model B (very flexible)	6	14	8	12

For a model whose predictions at x_0 across training sets are $\hat{y}_1, \dots, \hat{y}_4$ with mean $\bar{y}$, use these definitions:

$$\text{bias} = \bar{y} - f(x_0), \qquad \text{variance} = \tfrac{1}{4}\textstyle\sum_{i=1}^{4}(\hat{y}_i - \bar{y})^2, \qquad \text{expected squared error} \approx \text{bias}^2 + \text{variance} + \sigma^2,$$

where σ^2 is the variance of the label noise (irreducible; the same for every model).

(a) For each model compute the mean prediction, bias^2, and variance. Which model is "high bias" and which is "high variance"? Note that each is *low* in the other quantity.

	mean $\bar{y}$	bias^2	variance	high bias or high variance?
Model A	_____	_____	_____	_____
Model B	_____	_____	_____	_____

(b) With $\sigma^2 = 3$, compute the expected squared error of each model at x_0 and state which model you would deploy. Would your choice change for any value of σ^2? One sentence.

Expected squared error — A: _____ B: _____ Deploy: _____

(c) The table below is a *learning curve* — mean squared error on the training set and on a held-out validation set as the training-set size n grows — for two other models, X and Y (all four columns come from the same data-generating process).

n	X: train	X: validation	Y: train	Y: validation
50	3.9	8.0	0.1	9.0
200	4.0	5.5	0.2	6.5
1000	4.1	4.5	0.3	4.5
5000	4.1	4.2	0.4	3.0

Diagnose each of X and Y as mainly high-bias or mainly high-variance, giving the feature of the curve that tells you. For each, say whether collecting five times more data is worth the money, and name one other remedy from Lecture 07.

X is mainly high _____ Y is mainly high _____

Solution

(a) **Model A:** $\bar{y} = (7+8+8+9)/4 = 8$; bias $= 8-10 = -2$, bias$^2 = 4$; variance $= (1+0+0+1)/4 = 0.5$. **Model B:** $\bar{y} = (6+14+8+12)/4 = 10$; bias $= 0$, bias$^2 = 0$; variance $= (16+16+4+4)/4 = 10$. A is high bias (systematically 2 below the truth) but low variance (predictions barely move between training sets); B is unbiased on average but high variance (its prediction swings from 6 to 14 depending on which training set it saw).

(b) Expected squared error: A: $4+0.5+3 = 7.5$; B: $0+10+3 = 13$. Deploy A — its systematic error costs less than B's instability. The choice does not depend on σ^2: the noise term is added equally to both, so it never changes which is smaller (4.5 vs 10 before noise). *Common mistake:* concluding B is better because its average prediction is exactly right — a single deployed model is one draw, not the average over training sets.

(c) **X is high bias:** training and validation error converge quickly to about 4 and then plateau together (gap ≈ 0.1 at $n = 5000$). More data will not help — the two curves have already met; the model cannot represent the pattern. Remedy: a more flexible model (more/other features, higher-order terms, less regularization). **Y is high variance:** training error is near zero throughout while validation error is far above it, and the gap is still closing at $n = 5000$ (0.4 vs 3.0). Here more data *is* worth it (validation error is still falling steeply);

other remedies from Lecture 07 are limiting model complexity via a hyperparameter or adding regularization.

Problem 2: Choosing a split, then knowing when to stop

Practices: entropy and Gini by hand, information gain, and why the greedy tree keeps growing.

A lender records, for ten past borrowers, whether they own their home, whether they are married, and whether they later **defaulted** (the label):

#	Home owner	Married	Default
1	Y	Y	N
2	Y	Y	N
3	Y	N	N
4	Y	N	N
5	N	Y	Y
6	N	Y	N
7	N	Y	N
8	N	N	Y
9	N	N	Y
10	N	N	Y

With p_i the fraction of class i at a node (and $0 \log_2 0 = 0$), use

$$\text{Entropy} = -\sum_i p_i \log_2 p_i, \qquad \text{Gini} = 1 - \sum_i p_i^2.$$

The collective impurity of k children is their size-weighted average $I(\text{children}) = \sum_{j=1}^{k} \frac{N(v_j)}{N} I(v_j)$, and the information gain of a split is $I(\text{parent}) - I(\text{children})$. You may use $\log_2 3 \approx 1.585$ and $\log_2 5 \approx 2.322$; every other logarithm you need follows from these (e.g. $\log_2 \frac{2}{5} = 1 - \log_2 5$). Round to 3 decimals.

(a) Compute the entropy and the Gini index of the root node (all ten rows).

Entropy: _____ Gini: _____

(b) Two candidate splits are available at the root: on *Home owner* and on *Married*. For each, compute the collective (weighted) entropy of the two children and the information gain. Which split does the greedy algorithm choose? Check that the Gini gains agree on the ordering (compute both weighted Gini values).

split	collective entropy	information gain	weighted Gini	Gini gain
Home owner	_____	_____	_____	_____
Married	_____	_____	_____	_____

Greedy choice: _____

(c) Grow the tree one more level: on the impure child from (b), split on the remaining attribute, and compute its information gain. Write out the resulting two-level tree (the leaf labels are the majority class in each leaf) and its training accuracy on these ten rows.

Gain of the second split: _____ Training accuracy: _____

(d) One leaf of your tree is still impure. Explain in two or three sentences why a tree grown to purity on a real dataset (with more attributes to split on) tends to overfit, and say what each of the following would do to the tree in (c): `max_depth = 1`; `min_samples_leaf = 3` (a leaf must contain at least 3 training rows).

Solution

(a) Root: 4 Default = Y, 6 Default = N, so $p = (0.4, 0.6)$. $\log_2 0.4 = 1 - 2.322 = -1.322$, $\log_2 0.6 = 1.585 - 2.322 = -0.737$. Entropy $= -(0.4)(-1.322) - (0.6)(-0.737) = 0.529 + 0.442 = 0.971$. Gini $= 1 - 0.16 - 0.36 = 0.480$.

(b) **Home owner.** Y: rows 1–4, (0 Y, 4 N): entropy 0, Gini 0. N: rows 5–10, (4 Y, 2 N), $p = (\frac{2}{3}, \frac{1}{3})$: $\log_2 \frac{2}{3} = -0.585$, $\log_2 \frac{1}{3} = -1.585$; entropy $= 0.390 + 0.528 = 0.918$; Gini $= 1 - \frac{4}{9} - \frac{1}{9} = 0.444$. Collective entropy $= 0.4(0) + 0.6(0.918) = 0.551$; gain $= 0.971 - 0.551 = 0.420$. Weighted Gini $= 0.6(0.444) = 0.267$; Gini gain $= 0.213$. **Married.** Y: rows 1, 2, 5, 6, 7, (1 Y, 4 N), $p = (0.2, 0.8)$: $\log_2 0.2 = -2.322$, $\log_2 0.8 = -0.322$; entropy $= 0.464 + 0.258 = 0.722$; Gini $= 1 - 0.04 - 0.64 = 0.320$. N: rows 3, 4, 8, 9, 10, (3 Y, 2 N): entropy 0.971, Gini 0.480 (same proportions as the root, reversed). Collective entropy $= 0.5(0.722) + 0.5(0.971) = 0.846$; gain $= 0.125$. Weighted Gini $= 0.5(0.320) + 0.5(0.480) = 0.400$; Gini gain $= 0.080$. Choose **Home owner** (gain 0.420 vs 0.125); Gini agrees (0.213 vs 0.080).

(c) The impure child is Home owner = N (rows 5–10, entropy 0.918). Split on Married: Married = Y: rows 5, 6, 7, (1 Y, 2 N), entropy = 0.918; Married = N: rows 8, 9, 10, (3 Y, 0 N), entropy 0. Collective $= 0.5(0.918) + 0 = 0.459$; gain $= 0.918 - 0.459 = 0.459$. Tree: Home owner = Y $\rightarrow$ predict N; Home owner = N and Married = N $\rightarrow$ predict Y; Home owner = N and Married = Y $\rightarrow$ (1 Y, 2 N) majority N. Training accuracy: only row 5 is misclassified, so $9/10 = 90\%$ (the depth-1 stump gets 8/10).

(d) To make the (1 Y, 2 N) leaf pure the algorithm needs some attribute on which row 5 differs from rows 6 and 7 — with enough attributes (income, age, zip code, …) one always exists, so a fully grown tree carves out a leaf for every stray training row. Those tiny leaves record noise, not pattern; a different training sample would produce different leaves, which is exactly high variance. `max_depth = 1` stops after the Home owner split: the stump with 8/10 training accuracy, higher bias but more stable. `min_samples_leaf = 3` permits the tree in (c) (every leaf has 3 or 4 rows) but forbids any further split of the (1 Y, 2 N) leaf, since it would create a leaf with fewer than 3 rows — the guard bites exactly at the noisy leaf.

Problem 3: What averaging can and cannot buy

Practices: reasoning about ensembles with one identity, and separating bagging, random forests, and boosting by which error term they attack.

Suppose B predictors each have variance σ^2 at some input, and every pair has correlation ρ. The

variance of their average is

$$\mathrm{Var}\Big(\frac{1}{B}\sum_{b=1}^{B}\hat{y}_b\Big) = \rho\,\sigma^2 + \frac{(1-\rho)\,\sigma^2}{B}.$$

(a) Take $\sigma^2 = 1$. Compute the variance of the average for $(\rho, B) = (0.8, 10)$, $(0.8, 100)$, and $(0.2, 100)$. What does the variance approach as $B \to \infty$ at each ρ? In one sentence: what is the more effective lever, more trees or less correlated trees?

(ρ, B)	$(0.8, 10)$	$(0.8, 100)$	$(0.2, 100)$	limit, $\rho = 0.8$	limit, $\rho = 0.2$
variance of the average	_____	_____	_____	_____	_____

(b) Verify the identity for $B = 2$ using $\mathrm{Var}(X+Y) = \mathrm{Var}\,X + \mathrm{Var}\,Y + 2\,\mathrm{Cov}(X,Y)$, $\mathrm{Var}(aX) = a^2\,\mathrm{Var}\,X$, and $\mathrm{Cov}(X,Y) = \rho\,\sigma^2$ when both variances equal σ^2.

(c) *Bagging* grows each tree on a bootstrap sample of the rows and averages; a *random forest* does the same but, at every split, considers only a random subset of `max_features` of the attributes. In terms of ρ, σ^2, and B, say what each ingredient buys, and why the L08 activity found that bagging alone (all features at every split) barely improved on one deep tree when a few strong attributes dominate. What might a very small `max_features` cost you?

(d) *Boosting* (not covered in lecture — here is all you need) fits many *shallow* trees in sequence, each one trained on the errors the ensemble has made so far, and adds them up. Which of bias and variance is a shallow tree short on, so which term is boosting mainly reducing? Contrast with the forest in one sentence: why deep trees for a forest but shallow trees for boosting?

Solution

(a) $(0.8, 10)$: $0.8{+}0.2/10 = 0.820$. $(0.8, 100)$: $0.8{+}0.2/100 = 0.802$. $(0.2, 100)$: $0.2{+}0.8/100 = 0.208$. As $B \to \infty$ the second term vanishes and the variance tends to $\rho\sigma^2$: 0.8 and 0.2 respectively. Going from 10 to 100 trees at $\rho = 0.8$ shaved 0.018; dropping ρ from 0.8 to 0.2 cut the variance by four — decorrelating the trees is the lever, since the $\rho\sigma^2$ floor cannot be averaged away.

(b) $\mathrm{Var}\,(\frac{1}{2}(X+Y)) = \frac{1}{4}(\sigma^2+\sigma^2+2\rho\sigma^2) = \frac{\sigma^2(1+\rho)}{2}$. And $\rho\sigma^2 + \frac{(1-\rho)\sigma^2}{2} = \frac{2\rho\sigma^2+\sigma^2-\rho\sigma^2}{2} = \frac{\sigma^2(1+\rho)}{2}$. Equal.

(c) Bagging attacks the $(1-\rho)\sigma^2/B$ term: averaging B noisy, low-bias deep trees divides the *uncorrelated* part of their variance by B. But bootstrap samples of the same data still let the same few strong attributes win the top splits, so the trees look alike — ρ is high, the $\rho\sigma^2$ floor is high, and (as in (a)) more trees barely help. `max_features` attacks ρ: forcing splits to consider random attribute subsets makes different trees use different attributes, decorrelating them and lowering the floor — the ingredient that made the forest beat plain bagging. Cost: each tree is denied its best split more often, so its own error (bias, and a bit of σ^2) rises; too small a subset and the trees are individually poor — which is why `max_features` is tuned (about $\sqrt{p}$ by default).

(d) A shallow tree (a stump or depth 2–3) is high bias, low variance: it is too simple to fit the pattern but is stable. Boosting adds trees that each correct what the current sum still gets

wrong, so the sum becomes progressively more flexible — it reduces **bias**. A forest starts from low-bias, high-variance deep trees and averages away variance; boosting starts from high-bias, low-variance shallow trees and adds up bias-reducing corrections. Averaging shallow trees would fix nothing (their shared error is bias, and averaging cannot remove bias); boosting deep trees would have little bias left to remove and would overfit fast.